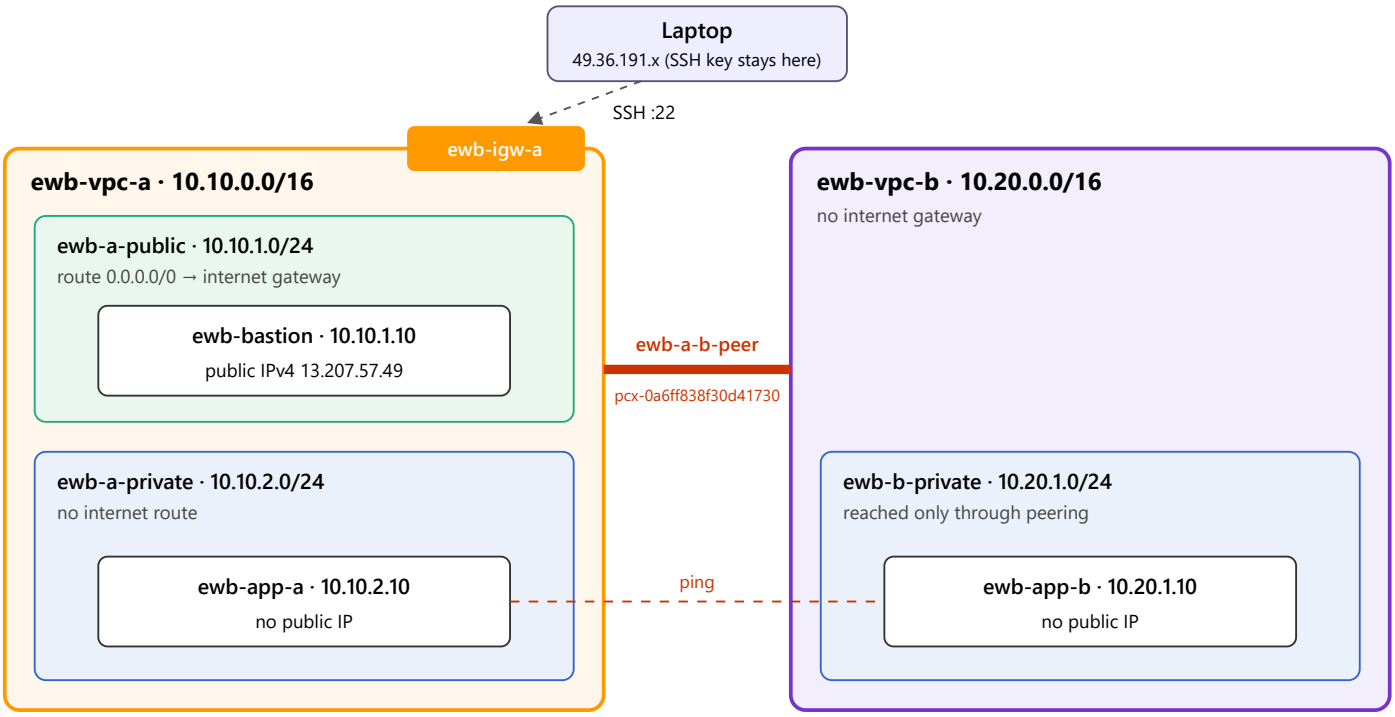
AWS VPC Peering — Private Connectivity Between Two VPCs

EWB Cloud Labs · AWS Architect Associate · Project 01

Name	Ayush Sharma
Batch	_____
Project title	AWS VPC Peering
Region	Asia Pacific (Mumbai) – ap-south-1 (AZ ap-south-1a)
Completion date	31 October 2026

1. Architecture



Administration path: laptop → bastion (public IP) → App-A or App-B, using SSH ProxyJump. The key stays on the laptop. **Project traffic:** App-A ↔ App-B over private IPs through the peering connection.

Resource inventory (actual IDs)

Resource	Name tag	ID	CIDR / IP
VPC A	ewb-vpc-a	vpc-02731d27d93f39c4e	10.10.0.0/16
Public subnet	ewb-a-public	subnet-0ab66f669901c2a1a	10.10.1.0/24
Private subnet A	ewb-a-private	subnet-0d472a371f713c1c3	10.10.2.0/24
VPC B	ewb-vpc-b	vpc-06d2fcab330b757a9	10.20.0.0/16
Private subnet B	ewb-b-private	subnet-04f07ce89fc29d57f	10.20.1.0/24
Internet gateway	ewb-igw-a	igw-038a952531790354a	attached to VPC A
Route table	ewb-rt-a-public	rtb-0dcff225d81e53af3	→ ewb-a-public
Route table	ewb-rt-a-private	rtb-01d35208e1131e3dd	→ ewb-a-private
Route table	ewb-rt-b-private	rtb-0554dcc5d61311b8e	→ ewb-b-private
Security group	ewb-sg-bastion	sg-08592da618a7838f0	VPC A
Security group	ewb-sg-app-a	sg-0c00f587850a4d498	VPC A
Security group	ewb-sg-app-b	sg-02ffbb97276e85651	VPC B
Peering connection	ewb-a-b-peer	pcx-0a6ff838f30d41730	A (requester) ↔ B (accepter)
EC2 instance	ewb-bastion	i-03260c4b0906d1ab8	10.10.1.10 + 13.207.57.49
EC2 instance	ewb-app-a	i-05853e589b8b14dce	10.10.2.10 (no public IP)
EC2 instance	ewb-app-b	i-07a5ef7328dc8182a	10.20.1.10 (no public IP)

AMI: Amazon Linux 2023 (ami-0f19324397f53458f), t3.micro, 8 GiB gp3 root volume (delete on termination), key pair `ewb-1ab`. Every resource is tagged Project = EWB-VPC-Peering and Student = Ayush Sharma.

2. Network: VPCs, subnets, internet gateway, route tables

VPCs: non-overlapping CIDRs 10.10.0.0/16 and 10.20.0.0/16

DescribeVpcs

Cidr	Id	Name
10.20.0.0/16	vpc-06d2fcab330b757a9	ewb-vpc-b
10.10.0.0/16	vpc-02731d27d93f39c4e	ewb-vpc-a

Subnets: auto-assign public IPv4 is disabled on all subnets. The bastion got its public IP at launch.

DescribeSubnets

AZ	AutoPublicIP	Cidr	Id	Name	Vpc
ap-south-1a	False	10.10.1.0/24	subnet-0ab66f669901c2a1a	ewb-a-public	vpc-02731d27d93f39c4e
ap-south-1a	False	10.20.1.0/24	subnet-04f07ce89fc29d57f	ewb-b-private	vpc-06d2fcab330b757a9
ap-south-1a	False	10.10.2.0/24	subnet-0d472a371f713c1c3	ewb-a-private	vpc-02731d27d93f39c4e

Internet gateway ewb-igw-a attached to VPC A

DescribeInternetGateways

AttachedVpc	Id	Name	State
vpc-02731d27d93f39c4e	igw-038a952531790354a	ewb-igw-a	available

All three route tables after peering: routes and explicit subnet associations

DescribeRouteTables

Id	Name	Routes	Subnets	Vpc
rtb-0554dcc5d61311b8e	ewb-rt-b-private	10.10.0.0/16-> pcx-0a6ff838f30d41730 10.20.0.0/16-> local	subnet-04f07ce89fc29d57f	vpc-06d2fcab330b757a9
rtb-0dcff225d81e53af3	ewb-rt-a-public	10.10.0.0/16-> local 10.20.0.0/16-> pcx-0a6ff838f30d41730 0.0.0.0/0-> igw-038a952531790354a	subnet-0ab66f669901c2a1a	vpc-02731d27d93f39c4e
rtb-01d35208e1131e3dd	ewb-rt-a-private	10.10.0.0/16-> local 10.20.0.0/16-> pcx-0a6ff838f30d41730	subnet-0d472a371f713c1c3	vpc-02731d27d93f39c4e

Why the public subnet needs a peering route: with ProxyJump, the bastion itself opens the TCP connection to App-B, so the bastion's subnet must route 10.20.0.0/16 to the pcx. The more specific /16 route wins over the 0.0.0.0/0 default route.

3. Peering and security groups

Peering ewb-a-b-peer: status ACTIVE, requester VPC A, accepter VPC B

DescribeVpcPeeringConnections

AccepterCidr	10.20.0.0/16
AccepterVpc	vpc-06d2fcab330b757a9
Id	pcx-0a6ff838f30d41730
Name	ewb-a-b-peer
RequesterCidr	10.10.0.0/16
RequesterVpc	vpc-02731d27d93f39c4e
Status	active

Inbound security group rules (outbound: default allow-all). Each source is a single /32 host.

DescribeSecurityGroupRules

Desc	Port / ICMP type	GroupId	Proto	Rule	Source	Port / ICMP code
Bastion private IP	22	sg-02ffbb97276e85651	tcp	sgr-01dc7f239d7333d2b	10.10.1.10/32	22
App-B echo request	8	sg-0c00f587850a4d498	icmp	sgr-067fd14f208679ea5	10.20.1.10/32	0
App-A echo request	8	sg-02ffbb97276e85651	icmp	sgr-08b4f61ce6ef1fccd	10.10.2.10/32	0
Bastion private IP	22	sg-0c00f587850a4d498	tcp	sgr-04516d92c658e6b0a	10.10.1.10/32	22
Student laptop	22	sg-08592da618a7838f0	tcp	sgr-06f00b8f8e8649c17	49.36.191.x/32	22

Group	Rule	Source	Reason
ewb-sg-bastion	TCP 22	49.36.191.x/32	Only my laptop's public IP can reach the bastion
ewb-sg-app-a	TCP 22	10.10.1.10/32	SSH only from the bastion
ewb-sg-app-a	ICMP echo (8/0)	10.20.1.10/32	Reverse ping from App-B
ewb-sg-app-b	TCP 22	10.10.1.10/32	Through ProxyJump, App-B sees the bastion's private IP as the source
ewb-sg-app-b	ICMP echo (8/0)	10.10.2.10/32	Forward ping from App-A

SSH configuration used on the laptop (~/.ssh/config)

```
Host ewb-bastion
    HostName 13.207.57.49
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
```

4. Verification tests

#	Test	Expected	Result
01	Baseline: App-A → 10.20.1.10 before peering	No replies	**100% loss (as expected)**
02	Forward: App-A → 10.20.1.10 after routes	4 replies, 0% loss	**4/4, 0% loss**
03	Reverse: App-B → 10.10.2.10 after routes	4 replies, 0% loss	**4/4, 0% loss**
04	Private SSH: laptop → ewb-app-b	Shell on App-B through bastion	**Connected as ec2-user**
05	Isolation: public IP on App-A / App-B	None	**None on either host**
06	Controlled failure: App-B ICMP rule removed / restored	Fail, then recover	**100% loss → 0% loss**

Laptop: ssh ewb-bastion (bastion reachable)

```
### HOST: ewb-bastion | 2026-10-31T12:51:23Z
$ ssh ewb-bastion 'hostname; hostname -I'
ip-10-10-1-10.ap-south-1.compute.internal
10.10.1.10
[exit 0]
```

TEST 01 / App-A / Baseline before peering: 100% packet loss

```
### HOST: ewb-app-a | 2026-10-31T12:51:24Z
$ ssh ewb-app-a 'hostname; hostname -I; ping -c 4 -W 2 10.20.1.10'
ip-10-10-2-10.ap-south-1.compute.internal
10.10.2.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3156ms

[exit 1]
```

Extra / Laptop: ssh ewb-app-b before peering: times out (no route from bastion to VPC B)

```
### HOST: ewb-app-b | 2026-10-31T12:51:32Z
$ ssh ewb-app-b 'hostname'
Connection timed out during banner exchange
[exit 255]
```

Extra / App-A / Peering ACTIVE but no routes yet: still 100% loss. Active peering alone is not enough.

```
### HOST: ewb-app-a | 2026-10-31T12:52:00Z
$ ssh ewb-app-a 'ping -c 4 -W 2 10.20.1.10'
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3110ms

[exit 1]
```

4. Verification tests (continued)

TEST 02 / App-A / Forward path ping 10.20.1.10: 4 received, 0% loss

```
### HOST: ewb-app-a | 2026-10-31T12:52:20Z
$ ssh ewb-app-a 'hostname; hostname -I; ping -c 4 10.20.1.10'
ip-10-10-2-10.ap-south-1.compute.internal
10.10.2.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.453 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.195 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.223 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.189 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3084ms
rtt min/avg/max/mdev = 0.189/0.265/0.453/0.109 ms
[exit 0]
```

TEST 03 / App-B (reached through the bastion) / Reverse path ping 10.10.2.10: 4 received, 0% loss

```
### HOST: ewb-app-b | 2026-10-31T12:52:26Z
$ ssh ewb-app-b 'hostname; hostname -I; ping -c 4 10.10.2.10'
ip-10-20-1-10.ap-south-1.compute.internal
10.20.1.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.192 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.215 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.226 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.217 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3140ms
rtt min/avg/max/mdev = 0.192/0.212/0.226/0.012 ms
[exit 0]
```

TEST 04 / Laptop: ssh ewb-app-b through ProxyJump: shell on App-B

```
### HOST: ewb-app-b | 2026-10-31T12:52:32Z
$ ssh ewb-app-b 'echo "Logged in to $(hostname) as $(whoami)"; hostname -I'
Logged in to ip-10-20-1-10.ap-south-1.compute.internal as ec2-user
10.20.1.10
[exit 0]
```

TEST 05 / EC2 instance details: App-A and App-B have no public IPv4

DescribeInstances

Id	Name	PrivateIP	PublicIP	SG	State	Subnet	Type
i-03260c4b0906d1ab8	ewb-bastion	10.10.1.10	13.207.57.49	ewb-sg-bastion	running	subnet-0ab66f669901c2a1a	t3.micro
i-07a5ef7328dc8182a	ewb-app-b	10.20.1.10	NONE	ewb-sg-app-b	running	subnet-04f07ce89fc29d57f	t3.micro
i-05853e589b8b14dce	ewb-app-a	10.10.2.10	NONE	ewb-sg-app-a	running	subnet-0d472a371f713c1c3	t3.micro

5. Controlled failure and recovery

Step 1: remove only App-B inbound ICMP Echo rule (source 10.10.2.10/32). SSH rule kept.

```
### Removing App-B inbound ICMP echo rule from 10.10.2.10/32 on sg-02ffbb97276e85651
{
    "Return": true,
    "RevokedSecurityGroupRules": [
        {
            "SecurityGroupRuleId": "sgr-05533444d1544f1b2",
            "GroupId": "sg-02ffbb97276e85651",
            "IsEgress": false,
            "IpProtocol": "icmp",
            "FromPort": 8,
            "ToPort": 0,
            "CidrIpv4": "10.10.2.10/32",
            "Description": "App-A echo request"
        }
    ]
}
```

Step 2 / App-A: ping 10.20.1.10: 100% packet loss

```
### HOST: ewb-app-a | 2026-10-31T12:52:41Z
$ ssh ewb-app-a 'ping -c 4 -W 2 10.20.1.10'
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3118ms

[exit 1]
```

Step 3: restore the exact rule

```
### Restoring the exact rule
```

AuthorizeSecurityGroupIngress

Code	Proto	Rule	Source	Type
0	icmp	sgr-08b4f61ce6ef1fccd	10.10.2.10/32	8

Step 4 / App-A: ping 10.20.1.10: 0% packet loss again

```
### HOST: ewb-app-a | 2026-10-31T12:52:56Z
$ ssh ewb-app-a 'ping -c 4 10.20.1.10'
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.192 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.197 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.250 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.294 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3098ms
rtt min/avg/max/mdev = 0.192/0.233/0.294/0.041 ms
[exit 0]
```

Explanation: routing and peering were unchanged during this test. Only the destination security group stopped permitting the ICMP echo request, so App-B silently dropped the packets. Restoring the /32 rule brought the replies back right away. This shows the routes give a path, but the security group decides whether traffic is accepted.

6. Forward path and return path

Forward (App-A → App-B): App-A (10.10.2.10) sends to 10.20.1.10. The ewb-rt-a-private table matches 10.20.0.0/16 → pcx-0a6ff838f30d41730. The packet crosses the peering connection into VPC B, and ewb-sg-app-b allows ICMP echo from 10.10.2.10/32.

Return (App-B → App-A): App-B's reply goes to 10.10.2.10. The ewb-rt-b-private table matches 10.10.0.0/16 → pcx-0a6ff838f30d41730, so the reply goes back across the peering connection. The security group is stateful, so the reply is allowed automatically.

Viva questions

1. Why is the return route needed? Routing is per subnet and one-way. Without 10.10.0.0/16 → pcx in VPC B, App-B's replies have no route back, so the connection fails even when the forward route exists.

2. Why is the App-B SSH source the bastion's private IP? With ProxyJump, the bastion opens a new TCP connection to App-B from its own interface (10.10.1.10). My laptop's IP never reaches App-B.

3. Why doesn't Active peering guarantee a successful ping? Peering only creates the link. Both route tables need routes to the pcx, and the destination security group must allow the traffic. Test "Peering ACTIVE but no routes yet" shows 100% loss.

4. Can VPC B use VPC A's internet gateway over peering? No. VPC peering doesn't support edge-to-edge routing: traffic from VPC B can't use VPC A's IGW, NAT or VPN. Peering is also non-transitive.

7. Cleanup

Cleanup actions

```
### 12:53:26Z  Terminate instances: i-03260c4b0906d1ab8 i-05853e589b8b14dce i-07a5ef7328dc8182a
i-07a5ef7328dc8182a     shutting-down
i-03260c4b0906d1ab8     shutting-down
i-05853e589b8b14dce     shutting-down
All three instances terminated

### 12:54:00Z  Delete key pair ewb-lab
True

### 12:54:02Z  Delete peer routes
ewb-rt-a-private 10.20.0.0/16 removed
ewb-rt-b-private 10.10.0.0/16 removed
ewb-rt-a-public 10.20.0.0/16 removed

### 12:54:07Z  Delete peering pcx-0a6ff838f30d41730
True

### 12:54:09Z  Delete security groups
sg-0c00f587850a4d498
sg-02ffbb97276e85651
sg-08592da618a7838f0

### 12:54:14Z  Delete subnets
subnet-0ab66f669901c2a1a deleted
subnet-0d472a371f713c1c3 deleted
subnet-04f07ce89fc29d57f deleted

### 12:54:20Z  Delete custom route tables
rtb-0dcff225d81e53af3 deleted
rtb-01d35208e1131e3dd deleted
rtb-0554dcc5d61311b8e deleted

### 12:54:25Z  Detach and delete internet gateway igw-038a952531790354a
igw-038a952531790354a deleted

### 12:54:29Z  Delete VPCs
vpc-02731d27d93f39c4e deleted
vpc-06d2fcab330b757a9 deleted
ids.env archived to evidence/00_resource_ids.txt
```

Verification: no project-tagged resources remain in ap-south-1

```
### 2026-10-31T12:54:36Z  Region ap-south-1, filter tag Project=EWB-VPC-Peering
Instances (not terminated): 0
EBS volumes:                0
Elastic IPs (any):          0
Key pair ewb-lab:           0
VPCs:                       0
Subnets:                    0
Route tables:               0
Internet gateways:          0
Security groups:            0
Peering (active/pending):   0
```

8. What I learned

A peering connection is only a link, not a path. After I accepted ewb-a-b-peer and it showed *active*, the ping from App-A to App-B still lost every packet. Traffic only started flowing once both private route tables had a route to the pcx. Before this lab I assumed "active" meant "connected".

Routing has to work in both directions. VPC A's route sends the echo request to VPC B, but the reply needs VPC B's own route (10.10.0.0/16 → pcx) to get back. One-way routing looks like a security group problem, because the packets just disappear.

The public subnet needed a peering route too. I didn't expect this at first. With ProxyJump, the bastion is the machine that opens the SSH connection to App-B. So ewb-rt-a-public needs 10.20.0.0/16 → pcx, and App-B's security group has to allow the bastion's private IP (10.10.1.10), not my laptop's IP. Before peering, `ssh ewb-app-b` simply timed out.

Security groups and routes do different jobs. In the controlled failure I removed only the ICMP rule on App-B. The routes and peering were untouched, but pings dropped to 0 replies, and they came back as soon as I restored the same /32 rule. Routes decide *where* traffic can go. Security groups decide *whether* it's accepted. Since security groups are stateful, I didn't need any rule for the replies.

Keeping things private on purpose. Only the bastion has a public IP, and its SSH rule allows just my own /32. App-A and App-B have no public address, and every rule names a single host instead of 0.0.0.0/0. The private key never left my laptop, because ProxyJump means I never had to copy it onto the bastion.

Peering is not transitive and doesn't share gateways. VPC B can talk to VPC A, but it can't use VPC A's internet gateway, and peering A–B plus A–C would not connect B to C. Non-overlapping CIDRs (10.10/16 and 10.20/16) were planned first, because peering can't be created between overlapping ranges.

Clean-up order matters. Instances had to be terminated before their security groups could be deleted, the peer routes before the peering connection, and the internet gateway detached before the VPC could go. Tagging every resource with Project = EWB-VPC-Peering made it easy to confirm that nothing was left running.